25VL612 DIGITAL VLSI TESTING AND TESTABILITY LAB

EXPERIMENT-6

Name : Vedasri J
Roll No : CB.EN.P2VLD25035
Date : 18/03/2026

EXPT NO: 6 A SCRIPT TO AUTOMATE T-MAX TOOL FLOW IN SYNOPSYS

Evaluation Table			Faculty Signature
Code:	Output:	Record:	

AIM:

To automate the **Synopsys T-Max ATPG tool flow** using a script for the **ISCAS-89** sequential benchmark circuit (s420.v), by performing scan cell insertion and Automatic Test Pattern Generation (ATPG), and to assess the effectiveness of scan-based testing by measuring fault coverage.

TOOLS USED:

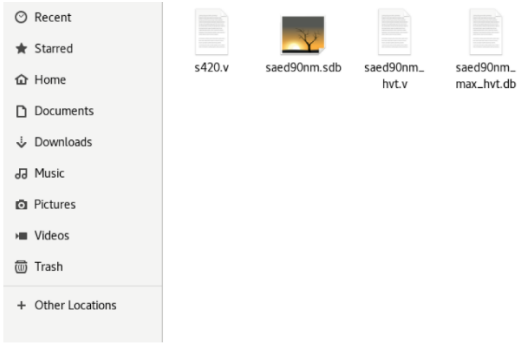
Synopsys DC Compiler, Synopsys TMax

PROCEDURE: STEPS INVOLVED IN AUTOMATING THE SYNOPSYS T-MAX TOOL FLOW

SYNOPSYS DC COMPILER

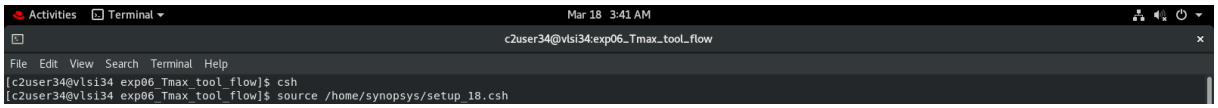
Step 1: Creating the Working Directory

A new working folder named **exp06_Tmax_tool_flow** was created inside the Linux environment, and the ISCAS-89 benchmark circuit file (**s420.v**) together with the SAED90nm library files were copied into it for use in the T-Max ATPG flow.



Step 2: Setting up the Synopsys Environment

The Synopsys environment was initialised by entering the **csh shell** and executing **source /home/synopsys/setup_18.csh**. This script exports all the necessary environment variables and tool paths for the Synopsys toolchain, including the Design Compiler and T-Max.



Step 3: Launching Design Compiler

The **Design Compiler (dc_shell)** was launched to synthesize the **s420 benchmark circuit** and produce its **gate-level netlist**, which is subsequently passed to T-Max for DFT insertion and ATPG.

```
Activities  Terminal                    Mar 18  3:41 AM
                          c2user34@vlsi34:exp06_Tmax_tool_flow
File  Edit  View  Search  Terminal  Help
[c2user34@vlsi34 exp06_Tmax_tool_flow]$ csh
[c2user34@vlsi34 exp06_Tmax_tool_flow]$ source /home/synopsys/setup_18.csh
[c2user34@vlsi34 exp06_Tmax_tool_flow]$ dc_shell

                         Design Compiler Graphical
                               DC Ultra (TM)
                                DFTMAX (TM)
                            Power Compiler (TM)
                               DesignWare (R)
                               DC Expert (TM)
                             Design Vision (TM)
                             HDL Compiler (TM)
                             VHDL Compiler (TM)
                                DFT Compiler
                             Design Compiler(R)

                 Version V-2023.12 for linux64 - Nov 27, 2023

                     Copyright (c) 1988 - 2023 Synopsys, Inc.
   This software and the associated documentation are proprietary to Synopsys,
 Inc. This software may only be used in accordance with the terms and conditions
 of a written license agreement with Synopsys, Inc. All other use, reproduction,
   or distribution of this software is strictly prohibited.  Licensed Products
     communicate with Synopsys servers for the purpose of providing software
    updates, detecting software piracy and verifying that customers are using
    Licensed Products in conformity with the applicable License Key for such
  Licensed Products. Synopsys will use information gathered in connection with
    this process to deliver software updates and pursue software pirates and
                                  infringers.

 Inclusivity & Diversity - Visit SolvNetPlus to read the "Synopsys Statement on
            Inclusivity and Diversity" (Refer to article 000036315 at
                         https://solvnetplus.synopsys.com)
Initializing...
Initializing gui preferences from file  /home/c2user34/.synopsys_dc_gui/preferences.tcl
Current time:        Wed Mar 18 03:41:27 2026
Hostname:            vlsi34.amritanet.edu
CPU Model:           Intel(R) Core(TM) i7-14700
CPU Details:         Cores = 28 : Sockets = 1 : Cache Size = 33792 KB : Freq = 2.10 GHz
OS:                  Linux 4.18.0-553.22.1.el8_10.x86_64
RAM:                  30 GB (Free  27 GB)
Swap:                 29 GB (Free  29 GB)
Work Filesystem:     / mounted to /dev/mapper/rhel-root
Tmp Filesystem:      / mounted to /dev/mapper/rhel-root
Work Disk:           158 GB (Free  33 GB)
Tmp Disk:            158 GB (Free  33 GB)

CPU Load: 1%, Ram Free: 27 GB, Swap Free: 29 GB, Work Disk Free: 33 GB, Tmp Disk Free: 33 GB
```

Step 4: Opening the Design Compiler GUI

The **Design Vision GUI** was opened by issuing **start_gui** within dc_shell. This graphical front-end provides a convenient interface for managing the design hierarchy and monitoring the compilation progress.

```
CPU Load: 1%, Ram Free: 27 GB, Swap Free: 29 GB, Work Disk Free: 33 GB, Tmp Disk Free: 33 GB
dc_shell> start_gui
dc_shell>
```

Step 5: Setting Technology Libraries

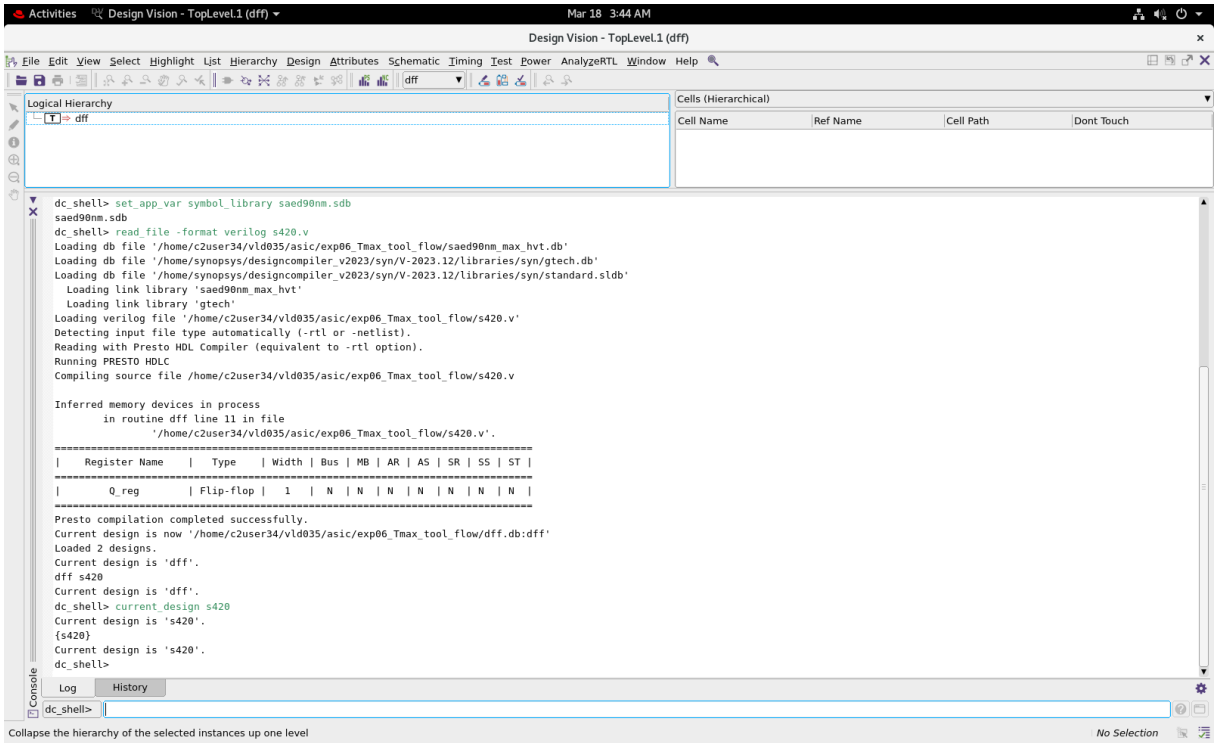
The **link library**, **target library**, and **symbol library** were configured to the SAED90nm library files (saed90nm_max_hvt.db and saed90nm.sdb) using set_app_var commands. These libraries provide the standard cells required for mapping the synthesized netlist.

```
Activities  Design Vision - TopLevel.1   Mar 18  3:44 AM
                          Design Vision - TopLevel.1
File  Edit  View  Select  Highlight  List  Hierarchy  Design  Attributes  Schematic  Timing  Test  Power  AnalyzeRTL  Window  Help
Logical Hierarchy                                   Cells (Hierarchical)
                                                    Cell Name | Ref Name | Cell Path | Dont Touch
CPU Model:           Intel(R) Core(TM) i7-14700
CPU Details:         Cores = 28 : Sockets = 1 : Cache Size = 33792 KB : Freq = 2.10 GHz
OS:                  Linux 4.18.0-553.22.1.el8_10.x86_64
RAM:                  30 GB (Free  27 GB)
Swap:                 29 GB (Free  29 GB)
Work Filesystem:     / mounted to /dev/mapper/rhel-root
Tmp Filesystem:      / mounted to /dev/mapper/rhel-root
Work Disk:           158 GB (Free  33 GB)
Tmp Disk:            158 GB (Free  33 GB)

CPU Load: 1%, Ram Free: 27 GB, Swap Free: 29 GB, Work Disk Free: 33 GB, Tmp Disk Free: 33 GB
dc_shell> start_gui
dc_shell> set_app_var link_path saed90nm_max_hvt.db
saed90nm_max_hvt.db
dc_shell> set_app_var target_library saed90nm_max_hvt.db
saed90nm_max_hvt.db
dc_shell> set_app_var symbol_library saed90nm.sdb
saed90nm.sdb
dc_shell>
Console   Log   History
dc_shell>
Ready                                                   No Selection
```

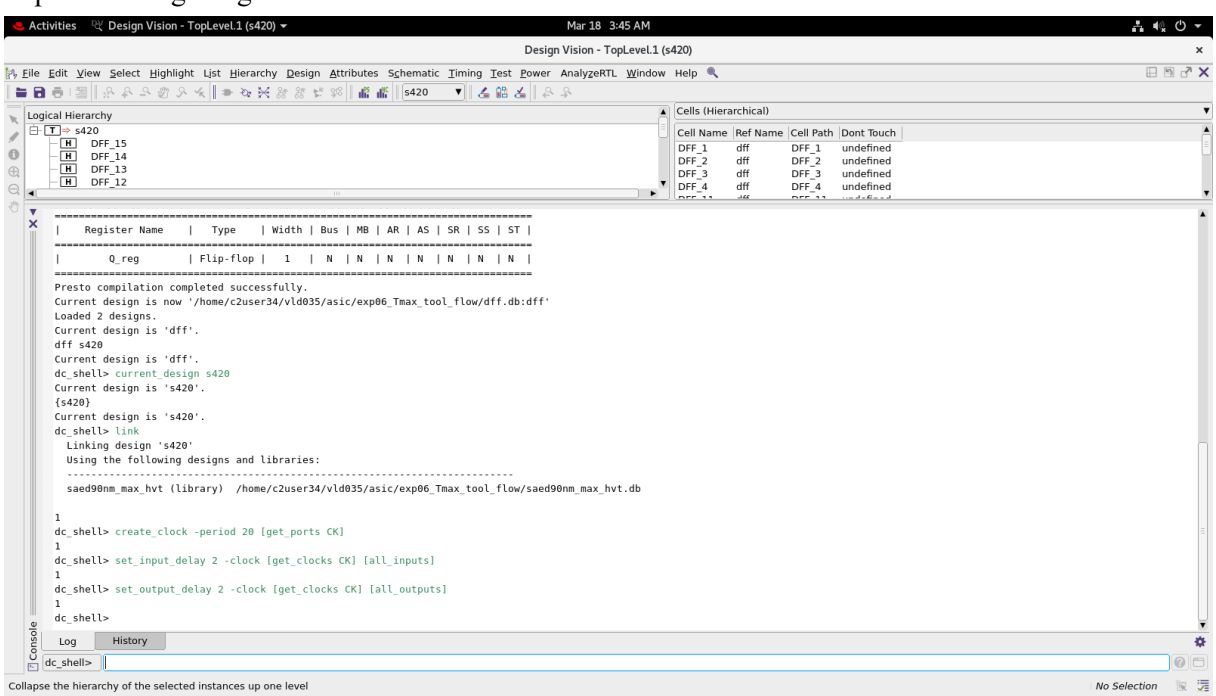
Step 6: Reading the Verilog File and Setting the Current Design

The s420 Verilog file was loaded into Design Compiler with **read_file -format verilog s420.v**. The top-level module was then designated as the active design with **current_design s420**, preparing the tool for synthesis and DFT insertion.



Step 7: Applying Timing Constraints

A clock of 20 ns period was created on port CK using **create_clock**. Input and output delays of 2 ns relative to this clock were then applied with **set_input_delay** and **set_output_delay**, guiding the synthesis tool to meet the required timing budget.



Step 8: Setting Scan Configuration

The scan cell type was set using **set_scan_configuration -style multiplexed_flip_flop**. This instructs Design Compiler to insert multiplexed flip-flops as scan cells, which is the preferred style for standard full-scan methodology.

```
dc_shell> set_output_delay 2 -clock [get_clocks CK] [all_outputs]
1
dc_shell> set_scan_configuration -style multiplexed_flip_flop
Accepted scan configuration for modes: all_dft
1
dc_shell>
```

Step 9: Defining the Scan Clock Signal

Port CK was registered as the scan clock with **set_dft_signal -view existing_dft -type ScanClock -port CK -timing [list 45 55]**. The timing values define the launch (45 ns) and capture (55 ns) edges within the scan test cycle.

```
dc_shell> set_dft_signal -view existing_dft -type ScanClock -port CK -timing [list 45 55]
Accepted dft signal specification for modes: all_dft
1
dc_shell>
```

Step 10: Compiling the Design with Scan Insertion

Synthesis with scan preparation was performed using **compile -scan -exact_map**. This command optimises the logic, maps it to SAED90nm cells, and internally annotates all flip-flops as scan candidates for the subsequent insert_dft step.

```
1
dc_shell> compile -scan -exact_map
CPU Load: 1%, Ram Free: 25 GB, Swap Free: 29 GB, Work Disk Free: 33 GB, Tmp Disk Free: 33 GB
Information: Choosing a test methodology will restrict the optimization of sequential cells. (UIO-12)
Information: Evaluating DesignWare library utilization. (UISN-27)

============================================================================
| DesignWare Building Block Library  |         Version         | Available |
============================================================================
| Basic DW Building Blocks           | V-2023.12-DWBB_202312.0 |     *     |
| Licensed DW Building Blocks        |                         |           |
============================================================================

==============================================================================================
| Flow Information                                                                           |
----------------------------------------------------------------------------------------------
| Flow         | Design Compiler                                                             |
| Command Line | -scan                                                                       |
==============================================================================================
| Design Information                                      | Value                            |
==============================================================================================
| Number of Scenarios                                     | 0                                |
| Leaf Cell Count                                         | 314                              |
| Number of User Hierarchies                              | 16                               |
| Sequential Cell Count                                   | 16                               |
| Macro Count                                             | 0                                |
| Number of Power Domains                                 | 0                                |
| Number of Path Groups                                   | 2                                |
| Number of VT Class                                      | 1                                |
| Number of Clocks                                        | 1                                |
| Number of Dont Touch Cells                              | 0                                |
| Number of Dont Touch Nets                               | 0                                |
| Number of Size Only Cells                               | 0                                |
| Design with UPF Data                                    | false                            |
==============================================================================================


  Beginning Pass 1 Mapping
  ------------------------
  Processing 'dff_0'
  Processing 's420'

  Updating timing information
Information: Updating design information... (UID-85)

  Beginning Mapping Optimizations  (Medium effort)
  ------------------------------

                                   TOTAL
   ELAPSED            WORST NEG    SETUP    DESIGN
    TIME      AREA      SLACK      COST    RULE COST         ENDPOINT
  --------- --------- --------- --------- --------- --------------------------
    0:00:00    1173.2      0.00       0.0       0.1
    0:00:00    1173.2      0.00       0.0       0.1
    0:00:00    1173.2      0.00       0.0       0.1
    0:00:00    1173.2      0.00       0.0       0.1
    0:00:00    1173.2      0.00       0.0       0.1
```

```
  0:00:00    1119.7      0.00       0.0       0.1
  0:00:00    1119.7      0.00       0.0       0.1
  0:00:00    1119.7      0.00       0.0       0.1
  0:00:00    1119.7      0.00       0.0       0.1
  0:00:00    1119.7      0.00       0.0       0.1
  0:00:00    1119.7      0.00       0.0       0.1
  0:00:00    1119.7      0.00       0.0       0.1


Beginning Delay Optimization Phase
----------------------------------

                                TOTAL
 ELAPSED            WORST NEG   SETUP    DESIGN
  TIME      AREA      SLACK     COST    RULE COST         ENDPOINT
--------- --------- --------- --------- --------- --------------------------
  0:00:00    1119.7      0.00       0.0       0.1
  0:00:00    1119.7      0.00       0.0       0.1
  0:00:00    1119.7      0.00       0.0       0.1
  0:00:00    1119.7      0.00       0.0       0.1
  0:00:00    1119.7      0.00       0.0       0.1

                                TOTAL
 ELAPSED            WORST NEG   SETUP    DESIGN
  TIME      AREA      SLACK     COST    RULE COST         ENDPOINT
--------- --------- --------- --------- --------- --------------------------
  0:00:00    1119.7      0.00       0.0       0.1


Beginning Area-Recovery Phase  (cleanup)
----------------------------

                                TOTAL
 ELAPSED            WORST NEG   SETUP    DESIGN
  TIME      AREA      SLACK     COST    RULE COST         ENDPOINT
--------- --------- --------- --------- --------- --------------------------
  0:00:00    1119.7      0.00       0.0       0.1
  0:00:00    1119.7      0.00       0.0       0.1
  0:00:00    1119.7      0.00       0.0       0.1
  0:00:00    1119.7      0.00       0.0       0.1
  0:00:00    1119.7      0.00       0.0       0.1
  0:00:00    1119.7      0.00       0.0       0.1
  0:00:00    1119.7      0.00       0.0       0.1
  0:00:00    1119.7      0.00       0.0       0.1
  0:00:00    1119.7      0.00       0.0       0.1
  0:00:00    1119.7      0.00       0.0       0.1
  0:00:00    1119.7      0.00       0.0       0.1
Loading db file '/home/c2user34/vld035/asic/exp06_Tmax_tool_flow/saed90nm_max_hvt.db'
Note: Symbol # after min delay cost means estimated hold TNS across all active scenarios


 Optimization Complete
 ---------------------
CPU Load: 1%, Ram Free: 25 GB, Swap Free: 29 GB, Work Disk Free: 33 GB, Tmp Disk Free: 33 GB
Information: Total number of MV cells in the design.
-------------------------------------------------------------------------------------------
 MV Cells                                                  Total Number
-------------------------------------------------------------------------------------------
 Level Shifter:                                            0
 Enable Level Shifter:                                     0
 Isolation Cell:                                           0
 Retention Cell:                                           0
 Retention Clamp Cell:                                     0
 Switch Cell:                                              0
 Always-On Cell:                                           0
 Repeater Cell:                                            0

-------------------------------------------------------------------------------------------
Unmapped MV Cells
-------------------------------------------------------------------------------------------
0 Isolation Cells are unmapped
0 Retention Clamp Cells are unmapped
-------------------------------------------------------------------------------------------
1
Current design is 's420'.
dc_shell>
```

Step 11: Creating the Test Protocol

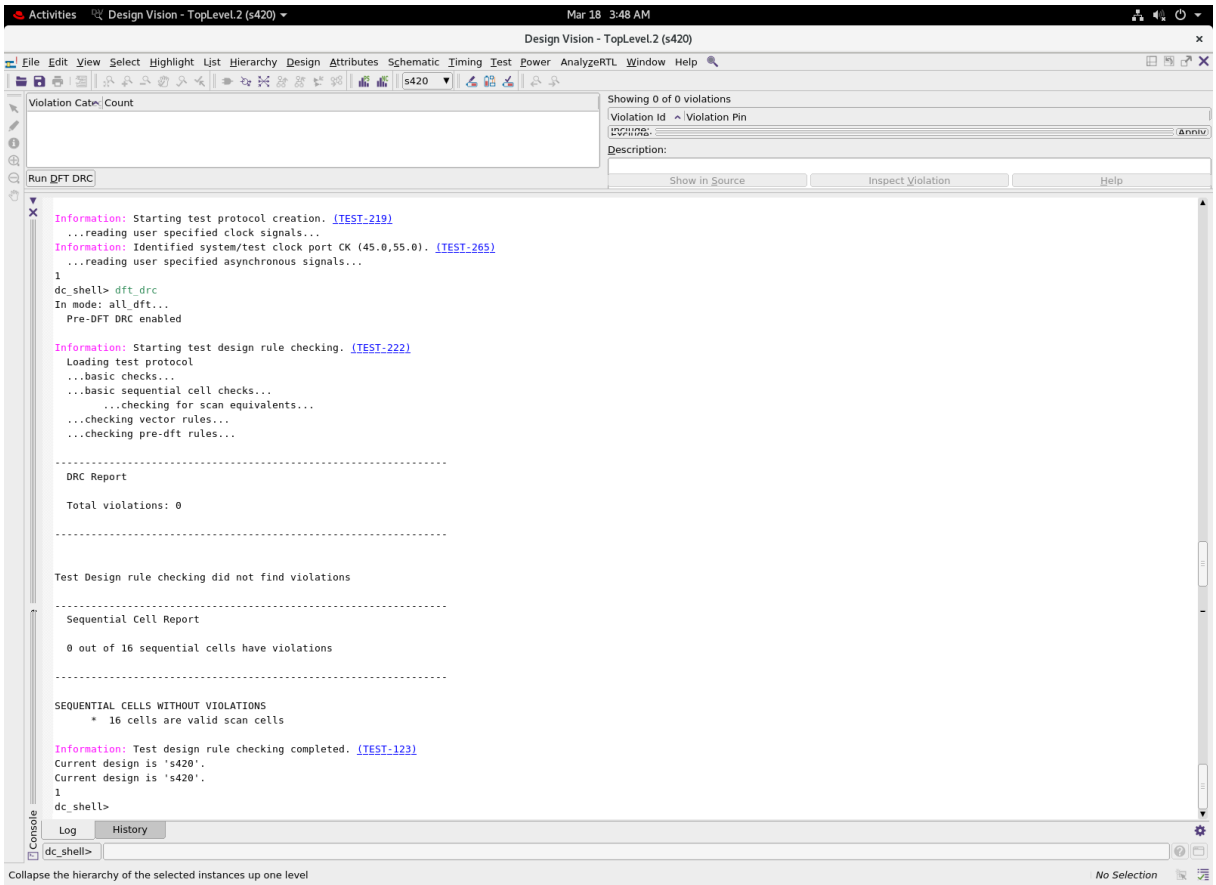
The test protocol was generated using **create_test_protocol**. This step reads the user-specified DFT signals and produces the internal representation of the scan test methodology, including clock and mode pin definitions.

```
dc_shell> create_test_protocol
In mode: all_dft...

Information: Starting test protocol creation. (TEST-219)
  ...reading user specified clock signals...
Information: Identified system/test clock port CK (45.0,55.0). (TEST-265)
  ...reading user specified asynchronous signals...
1
dc_shell>
```

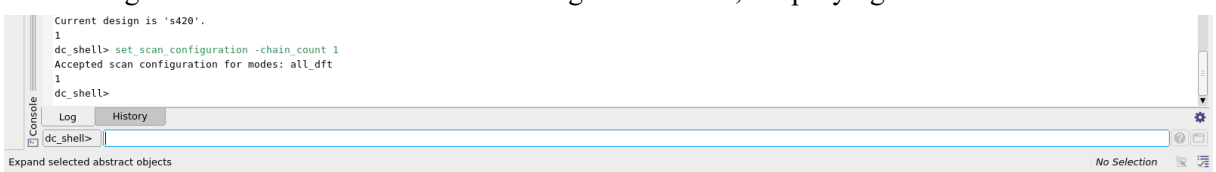
Step 12: Performing DFT Design Rule Check

A pre-insertion DFT design rule check was performed with **dft_drc**. The tool checked for scan equivalents, vector rules, and pre-DFT constraints, and confirmed **Total violations: 0**, meaning the design is fully compliant for scan insertion.



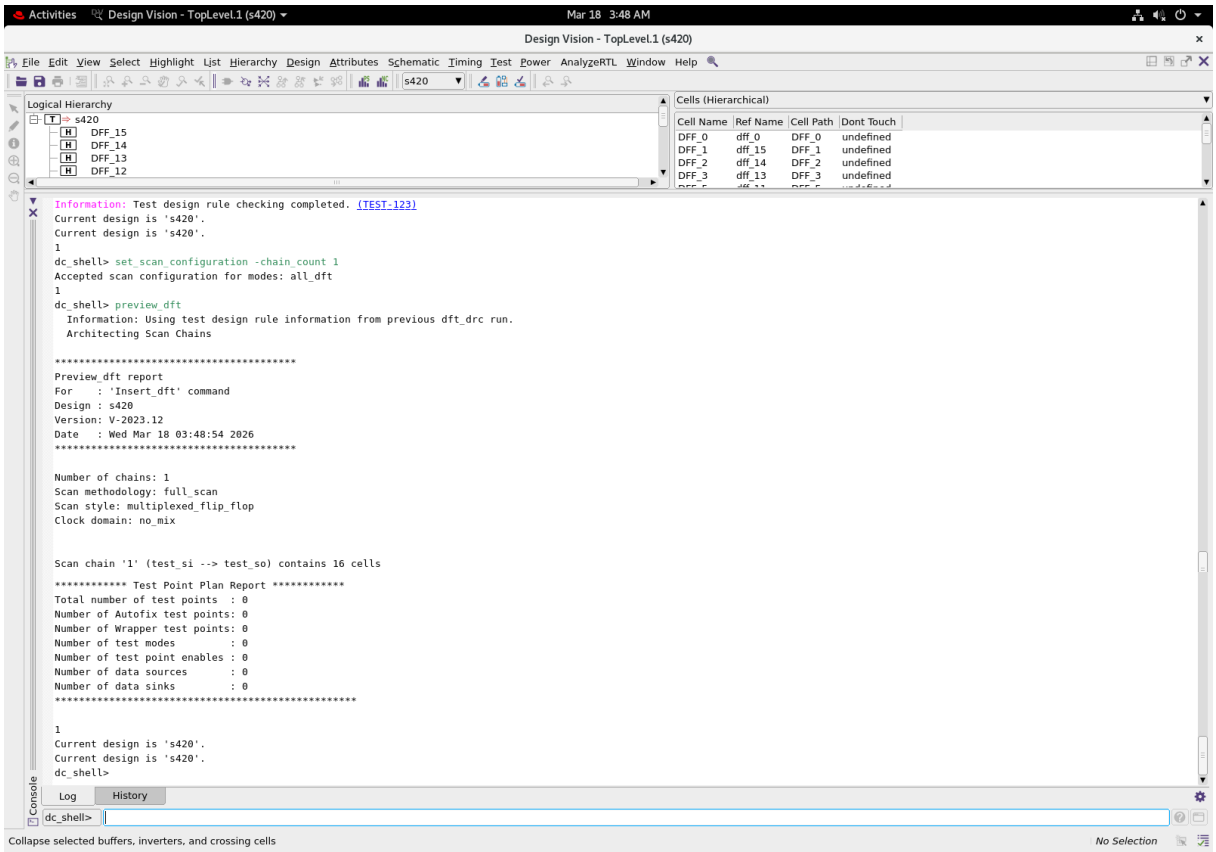
Step 13: Setting Scan Chain Count

The number of scan chains was fixed at 1 using **set_scan_configuration -chain_count 1**. All 16 sequential elements in the design will therefore be stitched into a single scan chain, simplifying the test interface.



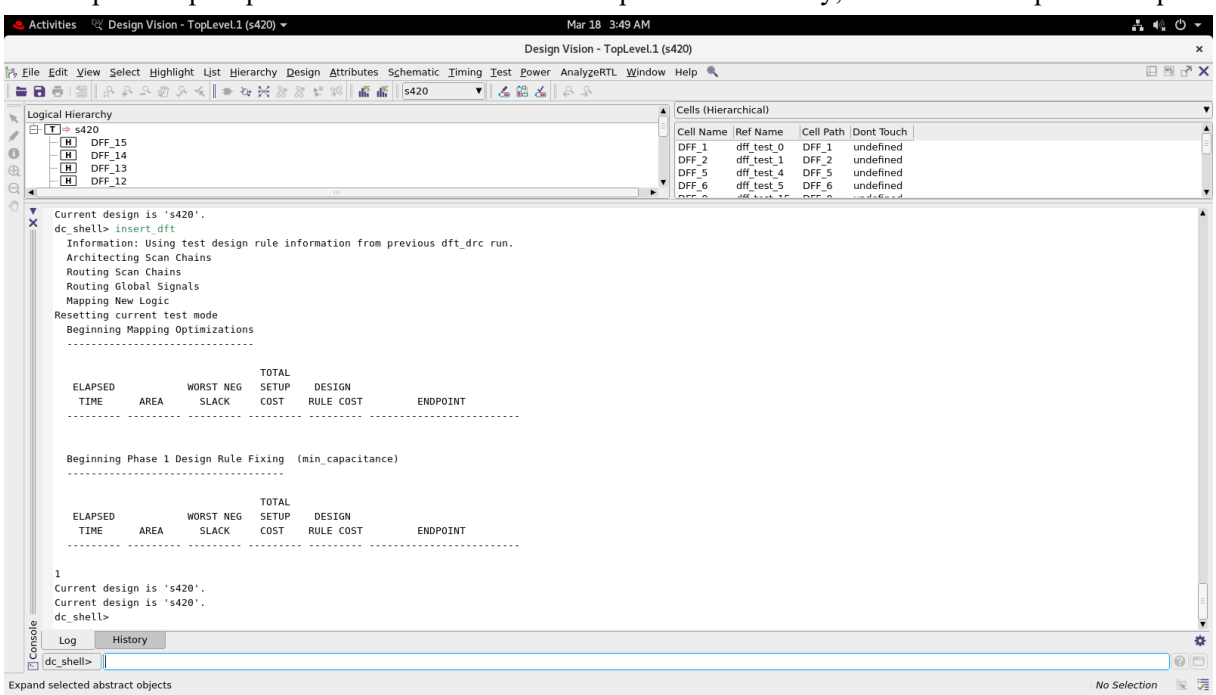
Step 14: Previewing the DFT Configuration

The **preview_dft** command produced a pre-insertion summary report. The report confirmed: 1 scan chain, full_scan methodology, multiplexed_flip_flop style, and a scan chain spanning 16 cells from test_si to test_so.



Step 15: Inserting Scan Chains

The scan chain was physically incorporated in the netlist using **insert_dft**. This step routes scan connections, maps scan-capable flip-flops to their scan-enabled counterparts in the library, and adds the required test ports.



Step 16: Updating Design Names and Writing the Netlist

Design names were updated for Verilog compliance using **change_names -hierarchy -rules verilog**. The scan-inserted gate-level netlist was then exported as **netlist_scan.v** using the write command with the -hierarchy flag to include all sub-modules.

```
1
Current design is 's420'.
Current design is 's420'.
dc_shell> change_names -hierarchy -rules verilog
1
dc_shell> write -format verilog -hierarchy -output netlist_scan.v
Writing verilog file '/home/c2user34/vld035/asic/exp06_Tmax_tool_flow/netlist_scan.v'.
1
dc_shell> write_test_protocol -output s420_scan_protocol.spf -names verilog
Writing test protocol file '/home/c2user34/vld035/asic/exp06_Tmax_tool_flow/s420_scan_protocol.spf' for mode 'Internal_scan'...
1
dc_shell>
```

Step 17: Writing the Test Protocol File

The SPF scan protocol file required by T-Max was produced by running **write_test_protocol -output s420_scan_protocol.spf -names verilog**. This file defines the scan procedures (load_unload, multiclock_capture) and clock configurations used during ATPG. Then Exit from dc compiler.

```
dc_shell> dc_shell> insert_dft
  Information: Using test design rule information from previous dft_drc run.
  Architecting Scan Chains
  Routing Scan Chains
  Routing Global Signals
  Mapping New Logic
Resetting current test mode
  Beginning Mapping Optimizations
  -------------------------------

                                  TOTAL
   ELAPSED            WORST NEG   SETUP    DESIGN
    TIME      AREA      SLACK     COST    RULE COST         ENDPOINT
  --------- --------- --------- --------- --------- -------------------------


  Beginning Phase 1 Design Rule Fixing  (min_capacitance)
  -----------------------------------

                                  TOTAL
   ELAPSED            WORST NEG   SETUP    DESIGN
    TIME      AREA      SLACK     COST    RULE COST         ENDPOINT
  --------- --------- --------- --------- --------- -------------------------

1
Current design is 's420'.
Current design is 's420'.
dc_shell> dc_shell> change_names -hierarchy -rules verilog
1
dc_shell> write -format verilog -hierarchy -output netlist_scan.v
Writing verilog file '/home/c2user34/vld035/asic/exp06_Tmax_tool_flow/netlist_scan.v'.
1
dc_shell> write_test_protocol -output s420_scan_protocol.spf -names verilog
Writing test protocol file '/home/c2user34/vld035/asic/exp06_Tmax_tool_flow/s420_scan_protocol.spf' for mode 'Internal_scan'...
1
dc_shell> exit

Memory usage for this session 164 Mbytes.
Memory usage for this session including child processes 164 Mbytes.
CPU usage for this session 6 seconds ( 0.00 hours ).
Elapsed time for this session 527 seconds ( 0.15 hours ).

Thank you...
[c2user34@vlsi34 exp06_Tmax_tool_flow]$
```

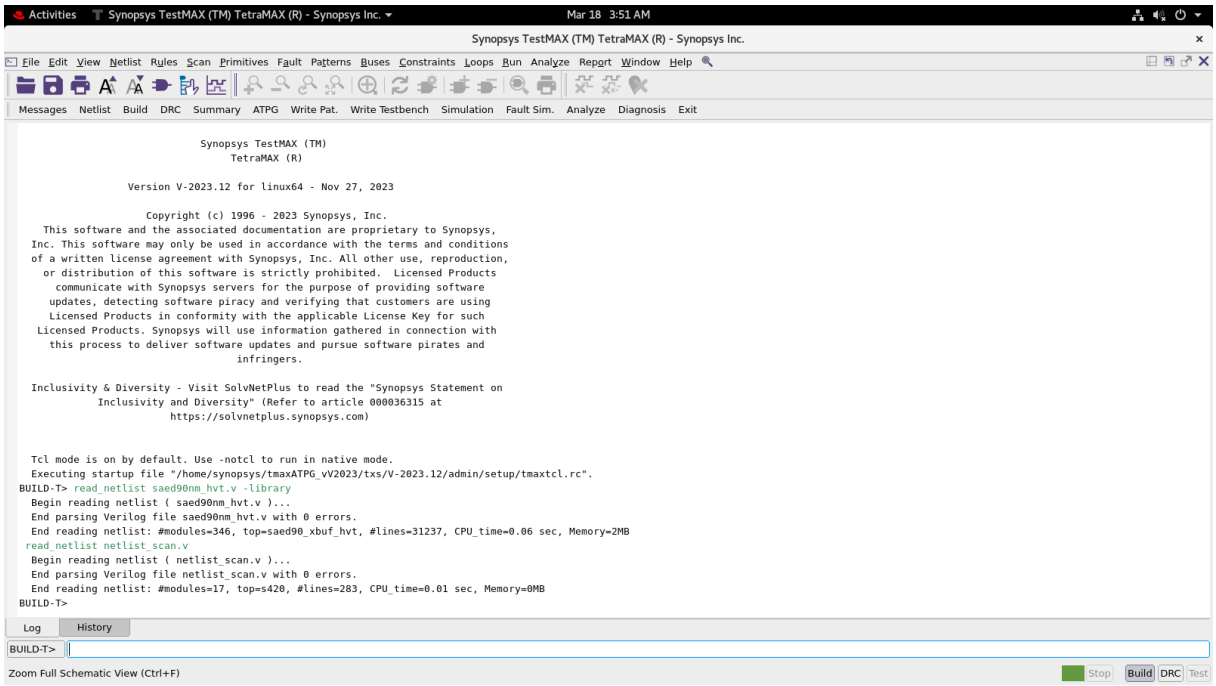
SYNOPSYS T-MAX

Step 18: Launching the T-Max Tool

After exiting dc_shell, the T-Max ATPG tool was started by running the command **tmax** in the terminal. The tool launched and displayed the BUILD-T prompt, indicating successful initialisation.

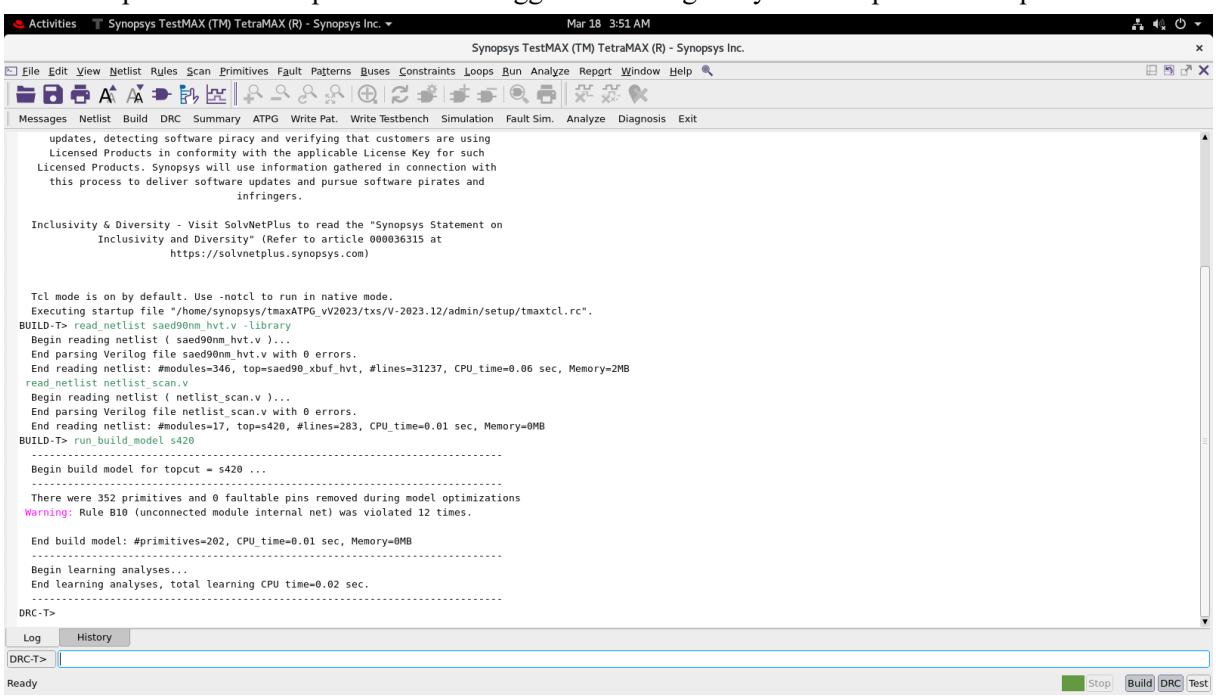
Step 19: Reading the Library and Design Netlist

The SAED90nm cell library (**saed90nm_hvt.v**) was loaded with the -library flag using **read_netlist**, followed by the scan-inserted design netlist (**netlist_scan.v**). Both files were parsed successfully with zero errors.



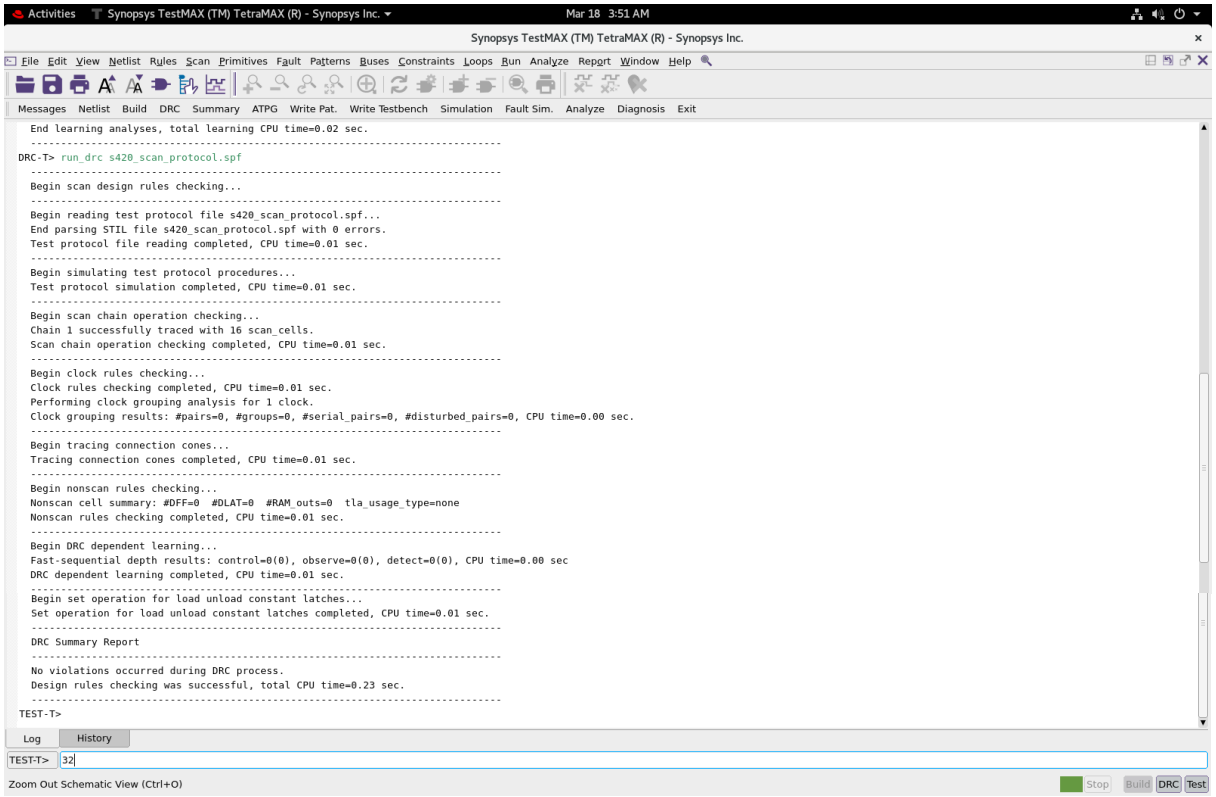
Step 20: Building the Test Model

The command **run_build_model s420** compiled the design into T-Max's internal fault model representation. The build completed with 202 primitives and triggered learning analyses to improve subsequent ATPG efficiency.



Step 21: Running DRC with the Test Protocol

The design rule check within T-Max was executed with **run_drc s420_scan_protocol.spf**. The tool verified scan chain tracing (16 scan cells found on Chain 1), clock grouping analysis, and non-scan cell rules, all passing without violations.



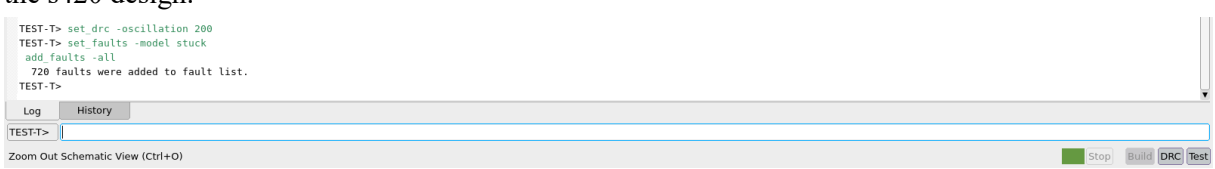
Step 22: Setting DRC Oscillation Limit and Fault Model

The oscillation limit was configured with **set_drc -oscillation 200** to bound the number of simulation cycles during DRC. The fault model was then set to stuck-at using **set_faults -model stuck**, targeting the most common manufacturing defects.

```
TEST-T> set_drc -oscillation 200
```

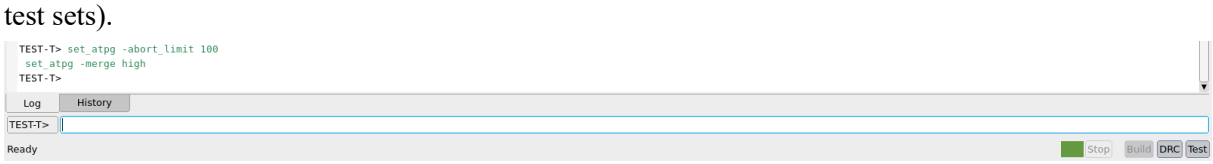
Step 23: Adding Faults to the Fault List

All stuck-at faults across the circuit were enumerated and loaded into the fault list with **add_faults -all**. T-Max reported that **720 faults were added** to the fault list, representing the complete set of uncollapsed stuck-at faults for the s420 design.



Step 24: Setting ATPG Parameters

ATPG settings were configured using **set_atpg -abort_limit 100** (limiting the maximum number of backtrack attempts per undetected fault) and **set_atpg -merge high** (activating high-effort pattern merging to achieve compact test sets).



Step 25: Running ATPG

Test pattern generation was performed with **run_atpg -auto_compression**. The tool employed deterministic ATPG to generate an initial set of patterns and then applied automatic compression, producing a final compact pattern set with full fault coverage.

```
TEST-T> run_atpg -auto_compression

  ATPG performed for stuck fault model using internal pattern source.
  ---------------------------------------------------------------------
  #patterns     #faults     #ATPG faults  test      process
  stored     detect/active  red/au/abort  coverage  CPU time
  ---------  -------------  ------------  --------  --------
  Begin deterministic ATPG: #uncollapsed_faults=601, abort_limit=100...
  30            509     92        0/0/0    87.22%      0.01
  60             92      0        0/0/0   100.00%      0.02

      Uncollapsed Stuck Fault Summary Report
  -----------------------------------------------
  fault class                     code   #faults
  ------------------------------  ----  ---------
  Detected                         DT        720
  Possibly detected                PT          0
  Undetectable                     UD          0
  ATPG untestable                  AU          0
  Not detected                     ND          0
  -----------------------------------------------
  total faults                                720
  test coverage                           100.00%
  -----------------------------------------------
            Pattern Summary Report
  -----------------------------------------------
  #internal patterns                           60
      #basic_scan patterns                     60
  -----------------------------------------------
            CPU Usage Summary Report
  -----------------------------------------------
  Total CPU time                            0.22
  -----------------------------------------------
TEST-T>
```

Step 26: Reporting Fault Summary

The **report_summaries** command displayed the final ATPG statistics. All 720 faults were classified as Detected (DT), with zero Possibly Detected (PT), Undetectable (UD), or Not Detected (ND) faults, verifying **100.00% test coverage**.

```
  Not detected                     ND          0
  -----------------------------------------------
  total faults                                720
  test coverage                           100.00%
  -----------------------------------------------
            Pattern Summary Report
  -----------------------------------------------
  #internal patterns                           60
      #basic_scan patterns                     60
  -----------------------------------------------
            CPU Usage Summary Report
  -----------------------------------------------
  Total CPU time                            0.22
  -----------------------------------------------
TEST-T> report_summaries
      Uncollapsed Stuck Fault Summary Report
  -----------------------------------------------
  fault class                     code   #faults
  ------------------------------  ----  ---------
  Detected                         DT        720
  Possibly detected                PT          0
  Undetectable                     UD          0
  ATPG untestable                  AU          0
  Not detected                     ND          0
  -----------------------------------------------
  total faults                                720
  test coverage                           100.00%
  -----------------------------------------------
            Pattern Summary Report
  -----------------------------------------------
  #internal patterns                           60
      #basic_scan patterns                     60
  -----------------------------------------------
TEST-T>
```

Step 27: Writing Test Patterns

The generated patterns were saved in STIL format using **write_patterns s420_patterns.stil -format stil -serial -replace**. T-Max reported 60 patterns written, 1099 test cycles generated, and a file size of 19134 bytes.

```
TEST-T> write_patterns s420_patterns.stil -format stil -serial -replace
  Patterns written reference 184 V statements, generating 1099 test cycles
  End writing file 's420_patterns.stil' with 60 patterns, File_size = 19134, CPU_time = 0.0 sec.
TEST-T>
```

Output:

Generated ATPG Test Patterns

The following show the s420_patterns.stil file output generated by T-Max for the ISCAS-89 benchmark circuit s420.v. Each pattern applies scan data through the test_si port and verifies the captured response on the test_so and primary output ports.

```
STIL 1.0 { Design 2005; }
Header {
   Title "  TetraMAX(R)  V-2023.12-i20231127_013657 STIL output";
   Date "Wed Mar 18 03:54:02 2026";
   Source "Minimal STIL for design `s420'";
   History {
      Ann {*  Incoming_Date "Wed Mar 18 03:45:35 2026"  *}
      Ann {*  Incoming_Src "DFT Compiler V-2023.12"  *}
      Ann {*     Uncollapsed Stuck Fault Summary Report *}
      Ann {* ---------------------------------------------- *}
      Ann {* fault class                     code   #faults *}
      Ann {* ------------------------------  ----  --------- *}
      Ann {* Detected                         DT        720 *}
      Ann {* Possibly detected                PT          0 *}
      Ann {* Undetectable                     UD          0 *}
      Ann {* ATPG untestable                  AU          0 *}
      Ann {* Not detected                     ND          0 *}
      Ann {* ---------------------------------------------- *}
      Ann {* total faults                               720 *}
      Ann {* test coverage                          100.00% *}
      Ann {* ---------------------------------------------- *}
      Ann {*  *}
      Ann {*            Pattern Summary Report *}
      Ann {* ---------------------------------------------- *}
      Ann {* #internal patterns                          60 *}
      Ann {*     #basic_scan patterns                    60 *}
      Ann {* ---------------------------------------------- *}
      Ann {*  *}
      Ann {* rule  severity  #fails  description *}
      Ann {* ----  --------  ------  ---------------------------------- *}
      Ann {* B10   warning       12  unconnected module internal net *}
      Ann {*  *}
      Ann {* clock_name        off  usage *}
      Ann {* ----------------  ---  -------------------------- *}
      Ann {* CK                 0   master shift  *}
      Ann {*  *}
      Ann {* There are no constraint ports *}
      Ann {* There are no equivalent pins *}
      Ann {* There are no net connections *}
      Ann {* top_module_name = s420 *}
      Ann {* Unified STIL Flow *}
      Ann {* serial_flag = 1 *}
      Ann {* PSDF = NO_PSD_FILE *}
      Ann {* PSDS = 0 *}
      Ann {* PSDA = #60#1#0/0 *}
      Ann {* internal_clock = CK *}
   }
}
Signals {
```

```
   "CK" In; "C_0" In; "C_1" In; "C_10" In; "C_11" In; "C_12" In; "C_13" In; "C_14" In;
   "C_15" In; "C_16" In; "C_2" In; "C_3" In; "C_4" In; "C_5" In; "C_6" In; "C_7" In;
   "C_8" In; "C_9" In; "P_0" In; "test_si" In { ScanIn; } "test_se" In; "Z" Out; "test_so" Out
   { ScanOut; }
}
SignalGroups {
   "_pi" = '"CK" + "C_0" + "C_1" + "C_10" + "C_11" + "C_12" + "C_13" + "C_14" + "C_15"
   + "C_16" + "C_2" + "C_3" + "C_4" + "C_5" + "C_6" + "C_7" + "C_8" + "C_9" + "P_0"
   + "test_si" + "test_se"'; // #signals=21
   "_in" = '"CK" + "C_0" + "C_1" + "C_10" + "C_11" + "C_12" + "C_13" + "C_14" + "C_15"
   + "C_16" + "C_2" + "C_3" + "C_4" + "C_5" + "C_6" + "C_7" + "C_8" + "C_9" + "P_0"
   + "test_si" + "test_se"'; // #signals=21
   "all_inputs" = '"CK" + "C_0" + "C_1" + "C_10" + "C_11" + "C_12" + "C_13" + "C_14"
   + "C_15" + "C_16" + "C_2" + "C_3" + "C_4" + "C_5" + "C_6" + "C_7" + "C_8" + "C_9"
   + "P_0" + "test_si" + "test_se"'; // #signals=21
   "_po" = '"Z" + "test_so"'; // #signals=2
   "_si" = '"test_si"' { ScanIn; } // #signals=1
   "all_outputs" = '"Z" + "test_so"'; // #signals=2
   "all_ports" = '"all_inputs" + "all_outputs"'; // #signals=23
   "_clk" = '"CK"'; // #signals=1
   "_so" = '"test_so"' { ScanOut; } // #signals=1
   "_out" = '"Z" + "test_so"'; // #signals=2
}
Timing {
   WaveformTable "_allclock_launch_capture_WFT_" {
      Period '100ns';
      Waveforms {
         "all_inputs" { 0 { '0ns' D; } }
         "all_inputs" { 1 { '0ns' U; } }
         "all_inputs" { Z { '0ns' Z; } }
         "all_inputs" { N { '0ns' N; } }
         "all_outputs" { X { '0ns' X; '40ns' X; } }
         "all_outputs" { H { '0ns' X; '40ns' H; } }
         "all_outputs" { L { '0ns' X; '40ns' L; } }
         "all_outputs" { T { '0ns' X; '40ns' T; } }
         "CK" { P { '0ns' D; '45ns' U; '55ns' D; } }
      }
   }
   WaveformTable "_multiclock_capture_WFT_" {
      Period '100ns';
      Waveforms {
         "all_inputs" { 0 { '0ns' D; } }
         "all_inputs" { 1 { '0ns' U; } }
         "all_inputs" { Z { '0ns' Z; } }
         "all_inputs" { N { '0ns' N; } }
         "all_outputs" { X { '0ns' X; '40ns' X; } }
         "all_outputs" { H { '0ns' X; '40ns' H; } }
         "all_outputs" { L { '0ns' X; '40ns' L; } }
         "all_outputs" { T { '0ns' X; '40ns' T; } }
         "CK" { P { '0ns' D; '45ns' U; '55ns' D; } }
      }
   }
   WaveformTable "_allclock_launch_WFT_" {
      Period '100ns';
      Waveforms {
         "all_inputs" { 0 { '0ns' D; } }
         "all_inputs" { 1 { '0ns' U; } }
```

```
            "all_inputs" { Z { '0ns' Z; } }
            "all_inputs" { N { '0ns' N; } }
            "all_outputs" { X { '0ns' X; '40ns' X; } }
            "all_outputs" { H { '0ns' X; '40ns' H; } }
            "all_outputs" { L { '0ns' X; '40ns' L; } }
            "all_outputs" { T { '0ns' X; '40ns' T; } }
            "CK" { P { '0ns' D; '45ns' U; '55ns' D; } }
         }
      }
      WaveformTable "_allclock_capture_WFT_" {
         Period '100ns';
         Waveforms {
            "all_inputs" { 0 { '0ns' D; } }
            "all_inputs" { 1 { '0ns' U; } }
            "all_inputs" { Z { '0ns' Z; } }
            "all_inputs" { N { '0ns' N; } }
            "all_outputs" { X { '0ns' X; '40ns' X; } }
            "all_outputs" { H { '0ns' X; '40ns' H; } }
            "all_outputs" { L { '0ns' X; '40ns' L; } }
            "all_outputs" { T { '0ns' X; '40ns' T; } }
            "CK" { P { '0ns' D; '45ns' U; '55ns' D; } }
         }
      }
      WaveformTable "_default_WFT_" {
         Period '100ns';
         Waveforms {
            "all_inputs" { 0 { '0ns' D; } }
            "all_inputs" { 1 { '0ns' U; } }
            "all_inputs" { Z { '0ns' Z; } }
            "all_inputs" { N { '0ns' N; } }
            "all_outputs" { X { '0ns' X; '40ns' X; } }
            "all_outputs" { H { '0ns' X; '40ns' H; } }
            "all_outputs" { L { '0ns' X; '40ns' L; } }
            "all_outputs" { T { '0ns' X; '40ns' T; } }
            "CK" { P { '0ns' D; '45ns' U; '55ns' D; } }
         }
      }
   }
   ScanStructures {
      ScanChain "1" {
         ScanLength 16;
         ScanIn "test_si";
         ScanOut "test_so";
         ScanInversion 0;
         ScanCellType  { CellIn "SI"; CellOut ! "QN"; }
         ScanCells { "s420.DFF_0.Q_reg" ; ! "s420.DFF_1.Q_reg" ; ! "s420.DFF_2.Q_reg"
         ; ! "s420.DFF_3.Q_reg" ; ! "s420.DFF_4.Q_reg" ; ! "s420.DFF_5.Q_reg" ; ! "s420.DFF_6.Q_reg"
         ; ! "s420.DFF_7.Q_reg" ; ! "s420.DFF_8.Q_reg" ; ! "s420.DFF_9.Q_reg" ; ! "s420.DFF_10.Q_reg"
         ; ! "s420.DFF_11.Q_reg" ; ! "s420.DFF_12.Q_reg" ; ! "s420.DFF_13.Q_reg" ; !
         "s420.DFF_14.Q_reg" ; ! "s420.DFF_15.Q_reg" ; !; }
         ScanMasterClock "CK" ;
      }
   }
   PatternBurst "_burst_" {
      PatList { "_pattern_" {
      }
   }}
```

```
PatternExec {
   PatternBurst "_burst_";
}
Procedures {
   "multiclock_capture" {
      W "_multiclock_capture_WFT_";
      C { "all_inputs"=0\r20 N ; "all_outputs"=XX; }
      V { "_pi"=\r21 # ; "_po"=##; }
   }
   "allclock_capture" {
      W "_allclock_capture_WFT_";
      C { "all_inputs"=0\r20 N ; "all_outputs"=XX; }
      V { "_pi"=\r21 # ; "_po"=##; }
   }
   "allclock_launch" {
      W "_allclock_launch_WFT_";
      C { "all_inputs"=0\r20 N ; "all_outputs"=XX; }
      V { "_pi"=\r21 # ; "_po"=##; }
   }
   "allclock_launch_capture" {
      W "_allclock_launch_capture_WFT_";
      C { "all_inputs"=0\r20 N ; "all_outputs"=XX; }
      V { "_pi"=\r21 # ; "_po"=##; }
   }
   "load_unload" {
      W "_default_WFT_";
      C { "all_inputs"=0\r20 N ; "all_outputs"=XX; }
      "Internal_scan_pre_shift": V { "test_se"=1; }
      Shift {          W "_default_WFT_";
         V { "_clk"=P; "_si"=#; "_so"=#; }
      }
   }
}
MacroDefs {
   "test_setup" {
      W "_default_WFT_";
      C { "all_inputs"=\r21 N ; "all_outputs"=XX; }
      V { "CK"=0; }
      V { }
   }
}
Pattern "_pattern_" {
   W "_multiclock_capture_WFT_";
   "precondition all Signals": C { "_pi"=\r21 0 ; "_po"=XX; }
   Macro "test_setup";
   Ann {* chain_test *}
   "pattern 0": Call "load_unload" {
      "test_si"=0011001100110011; }
   Call "multiclock_capture" {
      "_pi"=011011010101111001111; "_po"=HL; }
   "pattern 1": Call "load_unload" {
      "test_so"=LLHHLLHHLLHHLLHH; "test_si"=0100110011010101; }
   Call "multiclock_capture" {
      "_pi"=P10001101000010100110; "_po"=HL; }
   "pattern 2": Call "load_unload" {
      "test_so"=LHLLHHLLLHLHHLHL; "test_si"=1011111110101010; }
   Call "multiclock_capture" {
```

```
      "_pi"=P00101100000100111000; "_po"=LH; }
   "pattern 3": Call "load_unload" {
      "test_so"=HLHHHHHHHLHLHLHL; "test_si"=1011101110011010; }
   Call "multiclock_capture" {
      "_pi"=P01001100111111010100; "_po"=LH; }
   "pattern 4": Call "load_unload" {
      "test_so"=HLHHHLHHHLLHLLHL; "test_si"=1101000010111010; }
   Call "multiclock_capture" {
      "_pi"=P01111100111001111110; "_po"=HH; }
   "pattern 5": Call "load_unload" {
      "test_so"=HHLHLLLLHLHHLLHL; "test_si"=0101111001001101; }
   Call "multiclock_capture" {
      "_pi"=P00111110011100110110; "_po"=HL; }
   "pattern 6": Call "load_unload" {
      "test_so"=LHLHHHHLLHLLLHLH; "test_si"=1101101010101010; }
   Call "multiclock_capture" {
      "_pi"=P01011001100001111111; "_po"=LH; }
   "pattern 7": Call "load_unload" {
      "test_so"=HLHHLHLHLHLHLHLH; "test_si"=1010101010101010; }
   Call "multiclock_capture" {
      "_pi"=P00100110110000101111; "_po"=LH; }
   "pattern 8": Call "load_unload" {
      "test_so"=LHLHLHLHLHLHLHLH; "test_si"=1101111010101010; }
   Call "multiclock_capture" {
      "_pi"=P01001011110111011111; "_po"=LH; }
   "pattern 9": Call "load_unload" {
      "test_so"=HLHHHHLHLHLHLHLH; "test_si"=1100101010101010; }
   Call "multiclock_capture" {
      "_pi"=P00100111111011101101; "_po"=HH; }
   "pattern 10": Call "load_unload" {
      "test_so"=HLLHLHLHLHLHLHLL; "test_si"=0111101011111010; }
   Call "multiclock_capture" {
      "_pi"=P01010110111100110110; "_po"=LL; }
   "pattern 11": Call "load_unload" {
      "test_so"=LHHHHLHLHHHHLLHL; "test_si"=0001010111111101; }
   Call "multiclock_capture" {
      "_pi"=P00110011101001110100; "_po"=LL; }
   "pattern 12": Call "load_unload" {
      "test_so"=LLLHLHLHHHHHLHLH; "test_si"=0100101010101010; }
   Call "multiclock_capture" {
      "_pi"=P01000011010111110100; "_po"=LL; }
   "pattern 13": Call "load_unload" {
      "test_so"=LHLLHLHLHLHLLLHL; "test_si"=0111011101010101; }
   Call "multiclock_capture" {
      "_pi"=P00100001111011111110; "_po"=LL; }
   "pattern 14": Call "load_unload" {
      "test_so"=LHHHHLLHHLHLHLHL; "test_si"=1101101000110111; }
   Call "multiclock_capture" {
      "_pi"=P00001010011010110110; "_po"=LH; }
   "pattern 15": Call "load_unload" {
      "test_so"=HHLHHLHLLLHHHLLH; "test_si"=1100011010100010; }
   Call "multiclock_capture" {
      "_pi"=P00100101001101011100; "_po"=LH; }
   "pattern 16": Call "load_unload" {
      "test_so"=HHLLLHHLHLHLHHHL; "test_si"=0110010101110101; }
   Call "multiclock_capture" {
      "_pi"=P00101000001001100110; "_po"=LL; }
```

```
"pattern 17": Call "load_unload" {
   "test_so"=LHHLLHLHHLLHHLHL; "test_si"=0010101110010101; }
Call "multiclock_capture" {
   "_pi"=P01001110101011111110; "_po"=HL; }
"pattern 18": Call "load_unload" {
   "test_so"=LLHLHLHHLLLHHLHL; "test_si"=0110010001010101; }
Call "multiclock_capture" {
   "_pi"=P00100111101010111110; "_po"=LL; }
"pattern 19": Call "load_unload" {
   "test_so"=LHHLHLHHHLHLHLHL; "test_si"=1010101101010101; }
Call "multiclock_capture" {
   "_pi"=P10010011101010111110; "_po"=HH; }
"pattern 20": Call "load_unload" {
   "test_so"=HLHLLLHHHLHLHLHL; "test_si"=0110010101010101; }
Call "multiclock_capture" {
   "_pi"=P11001001110101011110; "_po"=HL; }
"pattern 21": Call "load_unload" {
   "test_so"=HLLLHLHLHLHLHLHL; "test_si"=1011100110111010; }
Call "multiclock_capture" {
   "_pi"=P01110100111010101111; "_po"=LH; }
"pattern 22": Call "load_unload" {
   "test_so"=LHHHLLHHLHHHLHLH; "test_si"=1001111010001110; }
Call "multiclock_capture" {
   "_pi"=P00101000100010111101; "_po"=LH; }
"pattern 23": Call "load_unload" {
   "test_so"=LLHHHHLHLLLHHHLL; "test_si"=1100001101010101; }
Call "multiclock_capture" {
   "_pi"=P01001110110110000110; "_po"=HH; }
"pattern 24": Call "load_unload" {
   "test_so"=HHLLHHHHHLHLHLHL; "test_si"=0110000000010101; }
Call "multiclock_capture" {
   "_pi"=P01111101110100001110; "_po"=HL; }
"pattern 25": Call "load_unload" {
   "test_so"=LHHLLLLLHHLHHLHL; "test_si"=0011111011010101; }
Call "multiclock_capture" {
   "_pi"=P00111110111010000010; "_po"=LL; }
"pattern 26": Call "load_unload" {
   "test_so"=LLHHHHHLHHLHLHLH; "test_si"=1001100110101010; }
Call "multiclock_capture" {
   "_pi"=P01101101110010011101; "_po"=LH; }
"pattern 27": Call "load_unload" {
   "test_so"=LLHHLLHHLHLHLHLL; "test_si"=1011101010101010; }
Call "multiclock_capture" {
   "_pi"=P01111101011001000110; "_po"=LH; }
"pattern 28": Call "load_unload" {
   "test_so"=HLHHHLHLHLHLLLHL; "test_si"=0101101110101001; }
Call "multiclock_capture" {
   "_pi"=P00101011110111101101; "_po"=LL; }
"pattern 29": Call "load_unload" {
   "test_so"=HLHHLHHHLHLHLLHL; "test_si"=0110111010001000; }
Call "multiclock_capture" {
   "_pi"=P01101010111100111111; "_po"=HL; }
"pattern 30": Call "load_unload" {
   "test_so"=HHLHHHLHLLLHLLLH; "test_si"=0000100001111010; }
Call "multiclock_capture" {
   "_pi"=P01100010010101111100; "_po"=LL; }
"pattern 31": Call "load_unload" {
```

```
      "test_so"=LLLLHLLLLHHHLLHL; "test_si"=1110110110101010; }
   Call "multiclock_capture" {
      "_pi"=P01010001001010111100; "_po"=LH; }
   "pattern 32": Call "load_unload" {
      "test_so"=HHHLHHLHHLHLLLHL; "test_si"=1000101110101010; }
   Call "multiclock_capture" {
      "_pi"=P01011010110000100110; "_po"=HH; }
   "pattern 33": Call "load_unload" {
      "test_so"=HLLLHLHHHLHLLLHL; "test_si"=0101010110101010; }
   Call "multiclock_capture" {
      "_pi"=P01101111001101100111; "_po"=LL; }
   "pattern 34": Call "load_unload" {
      "test_so"=HLHLHLHHLHLHLHLH; "test_si"=1001101010101010; }
   Call "multiclock_capture" {
      "_pi"=P01000100110011001101; "_po"=LH; }
   "pattern 35": Call "load_unload" {
      "test_so"=LLHHLHLHLHLHLHLL; "test_si"=1000101010101010; }
   Call "multiclock_capture" {
      "_pi"=P00100011111001100110; "_po"=HH; }
   "pattern 36": Call "load_unload" {
      "test_so"=HLLLHLHLHLHLLLHL; "test_si"=0001100010101010; }
   Call "multiclock_capture" {
      "_pi"=P00100111001001001111; "_po"=LL; }
   "pattern 37": Call "load_unload" {
      "test_so"=LLHHLLLHLHLHLHLH; "test_si"=0100001010101010; }
   Call "multiclock_capture" {
      "_pi"=P01011101100100100101; "_po"=LL; }
   "pattern 38": Call "load_unload" {
      "test_so"=HLLLLHLHLHLHLHLL; "test_si"=0100101110101010; }
   Call "multiclock_capture" {
      "_pi"=P01000110100111101110; "_po"=LL; }
   "pattern 39": Call "load_unload" {
      "test_so"=LHLLHLHHHLHLLLHL; "test_si"=0110011010101010; }
   Call "multiclock_capture" {
      "_pi"=P00100111010011111101; "_po"=HL; }
   "pattern 40": Call "load_unload" {
      "test_so"=HHLLHHLHLHLHLHLL; "test_si"=0010001010011010; }
   Call "multiclock_capture" {
      "_pi"=P00010011101001111110; "_po"=HL; }
   "pattern 41": Call "load_unload" {
      "test_so"=LLHLLLHLHLLHLLHL; "test_si"=0001100011001010; }
   Call "multiclock_capture" {
      "_pi"=P00101011100001000101; "_po"=HL; }
   "pattern 42": Call "load_unload" {
      "test_so"=LLHHLLLHHLLHLHLL; "test_si"=0110110001101010; }
   Call "multiclock_capture" {
      "_pi"=P00110111100111011110; "_po"=HL; }
   "pattern 43": Call "load_unload" {
      "test_so"=LHHLHHLLLHHLLLHL; "test_si"=0010011011111011; }
   Call "multiclock_capture" {
      "_pi"=P00011011110011101101; "_po"=LL; }
   "pattern 44": Call "load_unload" {
      "test_so"=LHLLHHLHHHHHLHHL; "test_si"=0001000100011011; }
   Call "multiclock_capture" {
      "_pi"=P00101111101100101110; "_po"=HL; }
   "pattern 45": Call "load_unload" {
      "test_so"=LLLHLLLHLLLHLLHH; "test_si"=1011101010101010; }
```

```
Call "multiclock_capture" {
   "_pi"=P01010111110110010111; "_po"=HH; }
"pattern 46": Call "load_unload" {
   "test_so"=LHHHLHLHLHLHLHLH; "test_si"=0110101010101010; }
Call "multiclock_capture" {
   "_pi"=P00001101101110110111; "_po"=HL; }
"pattern 47": Call "load_unload" {
   "test_so"=HHLHLHLHLHLHLHLH; "test_si"=1001100010101010; }
Call "multiclock_capture" {
   "_pi"=P00110110100010100101; "_po"=HH; }
"pattern 48": Call "load_unload" {
   "test_so"=LLHHLLLHLHLHLHLL; "test_si"=1101111010101010; }
Call "multiclock_capture" {
   "_pi"=P00110011010001010100; "_po"=HH; }
"pattern 49": Call "load_unload" {
   "test_so"=HHLHHHHLHLHLLLHL; "test_si"=0100001001110000; }
Call "multiclock_capture" {
   "_pi"=P00001001101000101000; "_po"=LL; }
"pattern 50": Call "load_unload" {
   "test_so"=LHLLLLHLLHHHLLLL; "test_si"=0100010101010101; }
Call "multiclock_capture" {
   "_pi"=P00000100110100010100; "_po"=LL; }
"pattern 51": Call "load_unload" {
   "test_so"=HLHHHLHLHLHLHLHL; "test_si"=0000000110001010; }
Call "multiclock_capture" {
   "_pi"=P00000010011010001110; "_po"=LL; }
"pattern 52": Call "load_unload" {
   "test_so"=LLLLLLLHHLLLLLHL; "test_si"=0100011000101010; }
Call "multiclock_capture" {
   "_pi"=P00100011011000111101; "_po"=LL; }
"pattern 53": Call "load_unload" {
   "test_so"=HLLLHHLLLHLHLHLL; "test_si"=1100000011101010; }
Call "multiclock_capture" {
   "_pi"=P00110011111000101110; "_po"=LH; }
"pattern 54": Call "load_unload" {
   "test_so"=HHLLLLLLHHHLLLHL; "test_si"=1101110000001000; }
Call "multiclock_capture" {
   "_pi"=P01111011100011001111; "_po"=LH; }
"pattern 55": Call "load_unload" {
   "test_so"=HLHHHLLLLLLHLLLH; "test_si"=1100010101010101; }
Call "multiclock_capture" {
   "_pi"=P11111101110100100100; "_po"=HH; }
"pattern 56": Call "load_unload" {
   "test_so"=LHLLHLHLHLHLHLHL; "test_si"=1101110001010101; }
Call "multiclock_capture" {
   "_pi"=P00011100101111101110; "_po"=LH; }
"pattern 57": Call "load_unload" {
   "test_so"=HHLHLHLLHLHLHLHL; "test_si"=0101111001000101; }
Call "multiclock_capture" {
   "_pi"=P10001110010111110100; "_po"=HL; }
"pattern 58": Call "load_unload" {
   "test_so"=LHLHHHHLHLHHHLHL; "test_si"=1110000010000100; }
Call "multiclock_capture" {
   "_pi"=P00100010111110000110; "_po"=LH; }
"pattern 59": Call "load_unload" {
   "test_so"=HHHLLLLLHLLLHLHH; "test_si"=0011010101010101; }
Call "multiclock_capture" {
```

```
      "_pi"=P00010010101111000100; "_po"=LL; }
   "end 59 unload": Call "load_unload" {
      "test_so"=HHHHHLHLHLHLHLHL; }
}

// Patterns reference 184 V statements, generating 1099 test cycles
```

Inference:

This experiment demonstrated the complete T-Max ATPG flow on the s420 ISCAS-89 sequential benchmark circuit. The design was first synthesized using Synopsys DC Compiler with the SAED90nm technology library, and a single scan chain of 16 flip-flops was inserted to make the circuit fully testable. The resulting scan-enabled netlist and SPF protocol file were then loaded into the Synopsys T-Max tool.

With the stuck-at fault model applied, a total of 720 faults were identified and added to the fault list. The ATPG engine, run with auto-compression enabled, generated 60 compact test patterns that successfully detected all 720 faults, achieving 100% fault coverage with zero undetectable or unresolved faults.

The generated patterns were exported in STIL format, which includes the full scan chain structure, waveform table definitions, and the load-unload and multiclock-capture procedures used during testing. The ScanStructures section of the STIL file confirms that all 16 DFFs in the s420 design are correctly chained from test_si to test_so, with CK as the master scan clock.

The results confirm that the full-scan DFT methodology, combined with deterministic ATPG, is highly effective for achieving complete fault detection in sequential circuits. The structured T-Max flow provides a reliable and repeatable approach to test quality that is suitable for industrial use.